



Universidad Andrés Bello

TAREA 1

La taxonomía de Bloom

Ignacio Avaria Guerra

José Tomás Mackliff Gamarra

Paradigmas de la Programación

Introducción

La gestión y creación de instrumentos de evaluación, como los exámenes escritos, representa un desafío recurrente para los docentes. Asegurar la calidad, la variedad cognitiva según taxonomías como la de Bloom, y evitar la repetición de ítems año tras año, requiere herramientas eficientes. La tarea solicitada surge como respuesta a esta necesidad, buscando aplicar los principios de la programación orientada a objetos para desarrollar una solución práctica. Siendo la motivación principal el facilitar el proceso de elaboración de evaluaciones, garantizando el cumplimiento de criterios pedagógicos y administrativos específicos. Este documento tiene como objetivo principal describir detalladamente la solución de software desarrollada en C++ para la gestión de preguntas y generación de evaluaciones. Se busca exponer el diseño implementado, justificando las decisiones tomadas durante el proceso de desarrollo.

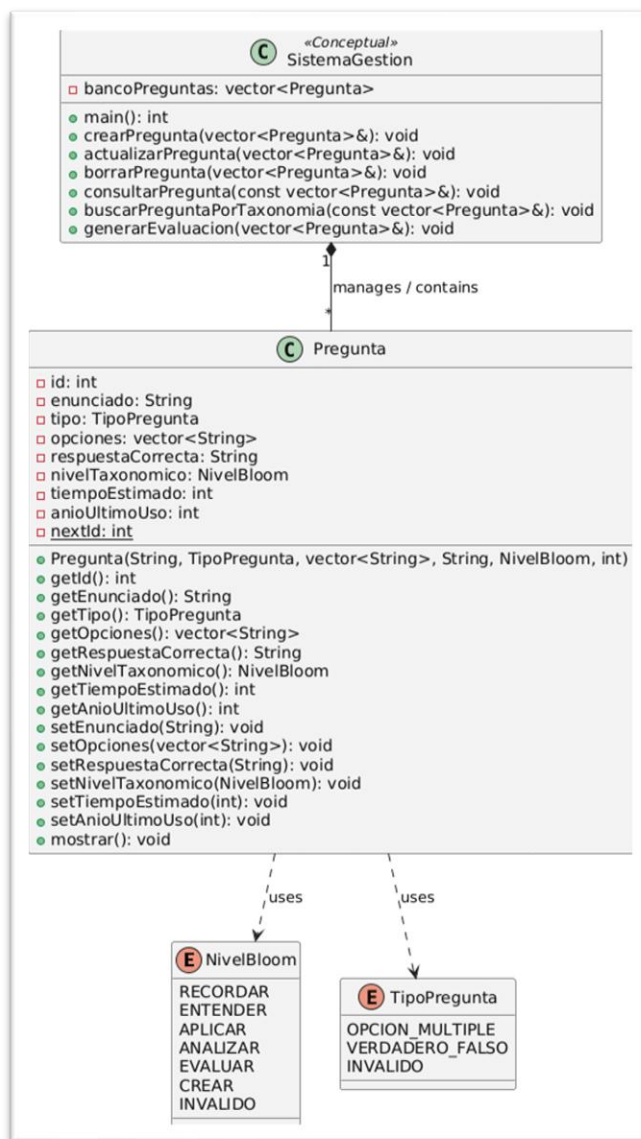


Diagrama de clases sobre el proyecto. Taxonomía de Bloom.

Descripción de la solución

Se ha desarrollado una aplicación de consola en C++ diseñada para gestionar un banco de preguntas y generar evaluaciones escritas. El sistema permite al usuario interactuar a través de un menú de texto, ofreciendo funcionalidades para crear, consultar, actualizar y eliminar preguntas (ítems), así como buscar preguntas por su nivel taxonómico según Bloom y generar evaluaciones personalizadas.

Clase Pregunta:

Es la entidad central del modelo. Representa una única pregunta de evaluación y encapsula toda su información relevante.

Atributos:

- Contiene id (entero único autoincremental).
- Enunciado (string).
- Tipo (enum TipoPregunta para diferenciar Opción Múltiple de Verdadero/Falso).
- Opciones (vector de strings, solo para Opción Múltiple).
- respuestaCorrecta (string).
- nivelTaxonomico (enum NivelBloom para clasificar según la taxonomía).
- tiempoEstimado (entero en minutos).
- anioUltimoUso (entero para controlar la regla de no repetición).

Un miembro estático nextId gestiona la generación de IDs únicos.

Métodos:

Incluye un constructor para inicializar objetos Pregunta, métodos get para acceder a los atributos, métodos set para modificarlos (usados en la actualización), y un método mostrar() para visualizar la información formateada de la pregunta en consola.

Enums (NivelBloom, TipoPregunta):

Se utilizan enum class para representar de forma segura y legible los distintos niveles de la taxonomía de Bloom y los tipos de pregunta permitidos, evitando el uso de strings o números "mágicos". Funciones auxiliares (...ToString, stringTo...) facilitan la conversión para la interfaz de usuario.

Almacenamiento (std::vector<Pregunta> bancoPreguntas):

Se optó por un std::vector en memoria dentro de main.cpp para almacenar la colección de objetos Pregunta. Este vector actúa como el "banco de preguntas" del sistema.

Funciones en main.cpp:

La lógica principal de la aplicación reside en funciones libres dentro de main.cpp. Estas incluyen:

- `main()`: Punto de entrada, contiene el bucle principal y el menú.
- Funciones CRUD: Implementan las operaciones básicas de gestión sobre el `bancoPreguntas`.
 - `crearPregunta`
 - `consultarPregunta`,
 - `actualizarPregunta`
 - `borrarPregunta`
- `buscarPreguntaPorTaxonomia`: Filtra y muestra preguntas según el nivel Bloom solicitado.
- `generarEvaluacion`: Orquesta la creación de una nueva evaluación según los criterios del usuario y las restricciones.
- Funciones auxiliares: Manejan la interacción con el usuario y la validación de entradas.
 - `mostrarMenu`
 - `obtenerOpcionValida`
 - `obtenerEnteroPositivo`
 - `obtenerEntero`

Discusión de Decisiones del Diseño:

Se utilizó C++ y una interfaz de consola según lo especificado en los requisitos de la tarea. Programando los distintos códigos utilizando las plataformas Replit y Dev-C++ para la facilitación del trabajo del programado. Se decidió modelar ambos tipos de pregunta (Opción Múltiple y V/F) por lo solicitado para la entrega, utilizando solo una clase `Pregunta`, diferenciándolas mediante el enum `TipoPregunta`. Esto se consideró más simple que usar herencia para solo dos tipos con atributos mayormente compartidos. El atributo `opciones` solo es relevante para Opción Múltiple. Se prefirió enum class sobre enum tradicional o constantes por la seguridad de tipos y claridad que ofrecen, mejorando la legibilidad y mantenibilidad del código. Además de la implementación de un contador estático (`nextId`) dentro de la clase `Pregunta` para asignar IDs únicos de forma automática y sencilla a cada nueva pregunta. Tomamos como elección `std::vector` para almacenar las preguntas en memoria por su simplicidad y adecuación al alcance de una tarea académica. Se reconoce que esta decisión implica que los datos se pierden al cerrar la aplicación; una mejora futura podría incluir persistencia en archivos (texto, CSV, JSON, etc.). Se optó por implementar la lógica del menú y las operaciones principales como funciones libres en main.cpp que operan sobre el vector `bancoPreguntas` (pasado por referencia). Aunque crear una clase `SistemaGestion` podría encapsular mejor la lógica y los datos, el enfoque procedural se consideró más directo para este proyecto específico. Mediante el atributo `anioUltimoUso` en `Pregunta`, la función `generarEvaluacion` obtiene el año actual usando `<chrono>` y `<ctime>` y filtra las preguntas candidatas asegurando que `anioUltimoUso < anioActual - 1`. Además, se utilizó `std::shuffle` sobre los índices de las preguntas candidatas válidas para evitar seleccionar siempre las mismas preguntas disponibles y añadir un factor de aleatoriedad en la generación de evaluaciones. Para finalizar se crearon funciones

auxiliares (`obtenerOpcionValida`, `obtenerEnteroPositivo`, `obtenerEntero`) para manejar la lectura de datos del usuario de forma robusta, limpiando el buffer de entrada y solicitando reintentos en caso de entradas inválidas.

Conclusión

El desarrollo del proyecto se abordó de manera incremental, comenzando por el diseño de la clase `Pregunta`, seguido por la implementación de la estructura del menú principal y la posterior codificación de cada una de las funcionalidades requeridas: creación, consulta, búsqueda, borrado, actualización y, finalmente, la generación de evaluaciones. Se prestó especial atención al manejo de la entrada/salida por consola y a la correcta aplicación de las restricciones, como la regla de no repetición anual.

El programa desarrollado cumple satisfactoriamente con todos los requisitos funcionales establecidos en el documento de la tarea. El sistema permite gestionar (crear, actualizar, borrar, consultar) ítems de tipo `Opción Múltiple` y `Verdadero/Falso`, clasifica cada ítem según la taxonomía de Bloom y un tiempo estimado, permite buscar ítems por taxonomía, y genera evaluaciones especificando el número y nivel de las preguntas, aplicando la restricción de no repetición anual y calculando el tiempo total estimado. La aplicación se ejecuta correctamente desde la consola de comandos. Además de que el programa está bien implementado con lo planteado en el Diagrama de Clases.

Para finalizar este proyecto ha sido una valiosa oportunidad para aplicar conceptos de programación orientada a objetos, manejo de memoria, entrada/salida, uso de la biblioteca estándar de C++, diseño de clases en la resolución de un problema práctico. El diseño de la estructura de datos (`Pregunta`, `enums`) y la implementación de la lógica de negocio (especialmente la generación de evaluaciones con sus restricciones) fueron los aspectos más desafiantes, complicados y que alimentaron más el conocimiento aprendido previamente en las clases. Se aprendió la importancia de la validación robusta de entradas de usuario y el manejo cuidadoso de los recursos (como el generador de números aleatorios o la obtención del tiempo del sistema). Como posible mejora futura, la persistencia de datos mediante archivos sería un paso lógico para hacer el sistema más útil en un escenario real. En general, la experiencia ha reforzado la comprensión de cómo traducir requisitos en una solución de software funcional y bien estructurada.